

# Manual Técnico y Arquitectura de Despliegue

**Plataforma:** RosterApp  
**Autor:** Ignacio Leo Sánchez  
**Módulo Académico:** 2º Desarrollo de Aplicaciones Web (DAW) - IES Albarregas

## 1. Paradigma y Arquitectura

RosterApp se ha construido bajo el paradigma de **Single Page Application (SPA)**. En esta arquitectura, el navegador carga un único documento HTML y un paquete JavaScript compilado. A medida que el usuario interactúa, JavaScript intercepta la navegación y actualiza el Modelo de Objetos del Documento (DOM) de forma dinámica, consumiendo datos a través de una API RESTful sin recargar la página. Para la infraestructura backend, se ha implementado un modelo **Backend as a Service (BaaS)** utilizando Supabase, que gestiona la base de datos PostgreSQL, la autenticación y las políticas de seguridad (Row Level Security).

## 2. Stack Tecnológico Frontend

| Tecnología   | Justificación y Rol en el Sistema                                                                                                                                                               |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| React (v18+) | Librería núcleo de renderizado. Facilita la creación de interfaces interactivas mediante componentes funcionales y la gestión del estado reactivo a través de Hooks.                            |
| TypeScript   | Superconjunto de JavaScript que provee tipado estático estricto. Garantiza la integridad de las interfaces de datos, minimiza los errores en tiempo de ejecución y mejora la mantenibilidad del |

| Tecnología              | Justificación y Rol en el Sistema                                                                                                                                                                                     |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                         | código.                                                                                                                                                                                                               |
| <b>Vite</b>             | Herramienta de compilación moderna que reemplaza a Webpack. Ofrece un <i>Hot Module Replacement (HMR)</i> ultrarrápido durante el desarrollo y optimiza el empaquetado final mediante Rollup.                         |
| <b>Tailwind CSS v4</b>  | Framework CSS <i>utility-first</i> en su última versión 4. Empleado por su nuevo motor de compilación que prescinde de dependencias complejas, proporcionando una escritura ágil y responsiva directamente sobre JSX. |
| <b>React Router DOM</b> | Gestor de navegación del lado del cliente. Crucial para definir el árbol de vistas, implementar <i>Layouts</i> maestros y crear componentes envolventes que protegen el acceso a rutas según el rol de autenticación. |

### 3. Árbol de Directorios y Modularidad

El código fuente se organiza siguiendo el patrón de carpetas por característica/funcionalidad, lo que favorece una arquitectura escalable:

```

rosterapp/
├── public/      # Assets públicos (favicon, logos) no procesados por Vite
├── src/        # Directorio raíz del código fuente
│   ├── components/  # Componentes React atómicos e independientes

```

|  |                   |                                                                   |
|--|-------------------|-------------------------------------------------------------------|
|  | └─ ui/            | # Componentes de interfaz puros (Botones, Inputs) estandarizados  |
|  | └─ layouts/       | # Envoltorios de diseño maestro (DashboardLayout, EmployeeLayout) |
|  | └─ lib/           | # Configuración de servicios de terceros y utilidades             |
|  | └─ supabase.ts    | # Instanciación y configuración del cliente de Supabase           |
|  | └─ utils.ts       | # Funciones helpers para formateo de fechas y clases              |
|  | └─ pages/         | # Módulos de página completos renderizados por el Router          |
|  | └─ admin/         | # Directorio de vistas del panel de control de gerencia           |
|  | └─ employee/      | # Directorio de vistas operativas del empleado base               |
|  | └─ App.tsx        | # Componente raíz donde se inicializa el enrutador                |
|  | └─ main.tsx       | # Punto de montaje del DOM (createRoot)                           |
|  | └─ .env           | # Fichero de credenciales locales (excluido de git)               |
|  | └─ vercel.json    | # Reglas Serverless para el entorno de producción                 |
|  | └─ vite.config.ts | # Ajustes del entorno de empaquetado de Vite                      |
|  | └─ package.json   | # Registro de dependencias y scripts NPM                          |

## 4. Guía de Instalación y Despliegue Local

1. **Requisitos Previos:** Es imperativo contar con Node.js (versión 18+) y un gestor de paquetes (npm o bun).
2. **Clonación y Configuración:** Tras clonar el repositorio, debe crear un archivo .env en el directorio raíz. En este archivo se deben declarar las variables que permiten la conexión con la base de datos de Supabase:  
VITE\_SUPABASE\_URL=https://[ID\_DEL\_PROYECTO].supabase.co  
VITE\_SUPABASE\_ANON\_KEY=[CLAVE\_PUBLICA\_ANONIMA\_JWT]
3. **Instalación de Dependencias:** Ejecute el comando npm install para construir la carpeta node\_modules.
4. **Arranque del Servidor:** Inicie el entorno de desarrollo con npm run dev. Vite compilará el código bajo demanda y levantará el proyecto (generalmente en localhost:5173).

## 5. Pipeline de Producción (Vercel)

El sistema está diseñado para un despliegue continuo (CI/CD) automatizado en **Vercel**. Al ser una SPA, si un usuario accede directamente o recarga una ruta anidada (ej. /employee/fichaje), el servidor en la nube de Vercel buscará un directorio físico y arrojará un error 404. Para evitar este comportamiento, se ha configurado un archivo vercel.json en la raíz del

proyecto que aplica una regla de *Rewrite*. Esto garantiza que cualquier solicitud de ruta sea redirigida internamente al archivo `index.html`, otorgando a React Router la capacidad de resolver y renderizar la vista correcta.

```
{
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

**Nota Final:** Para el correcto funcionamiento en producción, las variables `VITE_SUPABASE_URL` y `VITE_SUPABASE_ANON_KEY` deben ser inyectadas en la configuración de entorno (Environment Variables) del dashboard de Vercel antes de lanzar el comando de *build*.